Row- and column-major order

In computing, **row-major order** and **column-major order** are methods for storing multidimensional arrays in linear storage such as random access memory.

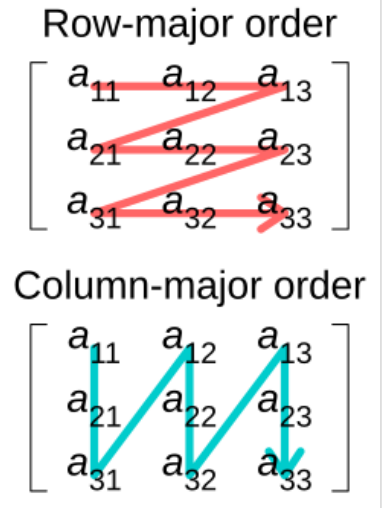

Illustration of difference between row- and column-major ordering

The difference between the orders lies in which elements of an array are contiguous in memory. In row-major order, the consecutive elements of a row reside next to each other, whereas the same holds true for consecutive elements of a column in column-major order. While the terms allude to the rows and columns of a two-dimensional array, i.e. a matrix, the orders can be generalized to arrays of any dimension by noting that the terms row-major and column-major are equivalent to lexicographic and colexicographic orders, respectively. Matrices, being commonly represented as collections of row or column vectors, using this approach are effectively stored as consecutive vectors or consecutive vector components. Such ways of storing data are referred to as AoS and SoA respectively.

Data layout is critical for correctly passing arrays between programs written in different programming languages. It is also important for performance when traversing an array because modern CPUs process sequential data more efficiently than nonsequential data. This is primarily due to CPU caching which exploits spatial locality of reference.[1] In addition, contiguous access makes it possible to use SIMD instructions that operate on vectors of data. In some media such as magnetic-tape data storage, accessing sequentially is orders of magnitude faster than nonsequential access.

Explanation and example

The terms row-major and column-major stem from the terminology related to ordering objects. A general way to order objects with many attributes is to first group and order them by one attribute, and then, within each such group, group and order them by another attribute, etc. If more than one attribute participates in ordering, the first would be called *major* and the last *minor*. If two attributes participate in ordering, it is sufficient to name only the major attribute.

In the case of arrays, the attributes are the indices along each dimension. For matrices in mathematical notation, the first index indicates the *row*, and the second indicates the *column*, e.g., given a matrix $\boldsymbol{A}$, the entry $\boldsymbol{a_{1,2}}$ is in its first row and second column. This convention is carried over to the syntax in programming languages,[2] although often with indexes starting at 0 instead of 1.[3]

Even though the row is indicated by the *first* index and the column by the *second* index, no grouping order between the dimensions is implied by this. The choice of how to group and order the indices, either by row-major or column-major methods, is thus a matter of convention. The same terminology can be

applied to even higher dimensional arrays. Row-major grouping starts from the *leftmost* index and column-major from the *rightmost* index, leading to lexicographic and colexicographic (or colex) orders, respectively.

For example, the array

$$A = a_{y,x} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{bmatrix}$$

could be stored in two possible ways:

Address	Row-major order	Column-major order
0	a_{11}	a_{11}
1	a_{12}	a_{21}
2	a_{13}	a_{12}
3	a_{21}	a_{22}
4	a_{22}	a_{13}
5	a_{23}	a_{23}

Programming languages handle this in different ways. In C, multidimensional arrays are stored in row-major order, and the array indexes are written row-first (lexicographical access order):

C: row-major order (lexicographical access order), zero-based indexing

Address `x + N_x*y`	Access `A[y][x]`	Value $a_{y,x}$
0	`A[0][0]`	a_{11}
1	`A[0][1]`	a_{12}
2	`A[0][2]`	a_{13}
3	`A[1][0]`	a_{21}
4	`A[1][1]`	a_{22}
5	`A[1][2]`	a_{23}

On the other hand, in Fortran, arrays are stored in column-major order, while the array indexes are still written row-first (colexicographical access order):

Fortran: column-major order (colexicographical access order), one-based indexing

Address `y + N_y*(x-1)`	Access `A(y,x)`	Value $a_{y,x}$
1	`A(1,1)`	a_{11}
2	`A(2,1)`	a_{21}
3	`A(1,2)`	a_{12}
4	`A(2,2)`	a_{22}
5	`A(1,3)`	a_{13}
6	`A(2,3)`	a_{23}

Note how the use of `A[i][j]` with multi-step indexing as in C, as opposed to a neutral notation like `A(i,j)` as in Fortran, almost inevitably implies row-major order for syntactic reasons, so to speak, because it can be rewritten as `(A[i])[j]`, and the `A[i]` row part can even be assigned to an intermediate variable that is then indexed in a separate expression. (No other implications should be assumed, e.g., Fortran is not column-major simply *because* of its notation, and even the above implication could intentionally be circumvented in a new language.)

To use column-major order in a row-major environment, or vice versa, for whatever reason, one workaround is to assign non-conventional roles to the indexes (using the first index for the column and the second index for the row), and another is to bypass language syntax by explicitly computing positions in a one-dimensional array. Of course, deviating from convention probably incurs a cost that increases with the degree of necessary interaction with conventional language features and other code, not only in the form of increased vulnerability to mistakes (forgetting to also invert matrix multiplication order, reverting to convention during code maintenance, etc.), but also in the form of having to actively rearrange elements, all of which have to be weighed against any original purpose such as increasing performance. Running the loop row-wise is preferred in row-major languages like C and vice versa for column-major languages.

Programming languages and libraries

Programming languages or their standard libraries that support multi-dimensional arrays typically have a native row-major or column-major storage order for these arrays.

Row-major order is used in C/C++/Objective-C (for C-style arrays), PL/I,[4] Pascal,[5] Speakeasy,[6] and SAS.[7]

Column-major order is used in Fortran,[8][9] IDL,[8] MATLAB,[9] GNU Octave, Julia,[10] S, S-PLUS,[11] R,[12] Scilab,[13] Yorick, and Rasdaman.[14]

Neither row-major nor column-major

A typical alternative for dense array storage is to use Iliffe vectors, which typically store pointers to elements in the same row contiguously (like row-major order), but not the rows themselves. They are used in (ordered by age): Java,[15] C#/CLI/.Net, Scala,[16] and Swift.

Even less dense is to use lists of lists, e.g., in Python,[17] and in the Wolfram Language of Wolfram Mathematica.[18]

An alternative approach uses tables of tables, e.g., in Lua.[19]

External libraries

Support for multi-dimensional arrays may also be provided by external libraries, which may even support arbitrary orderings, where each dimension has a stride value, and row-major or column-major are just two possible resulting interpretations.

Row-major order is the default in NumPy[20] (for Python).

Column-major order is the default in Eigen[21] and Armadillo (https://arma.sourceforge.net/docs.html) (both for C++).

A special case would be OpenGL (and OpenGL ES) for graphics processing. Since "recent mathematical treatments of linear algebra and related fields invariably treat vectors as columns," designer Mark Segal decided to substitute this for the convention in predecessor IRIS GL, which was to write vectors as rows; for compatibility, transformation matrices would still be stored in vector-major (row-major) rather than coordinate-major (column-major) order, and he then used the trick "[to] say that matrices in OpenGL are stored in column-major order".[22] This was really only relevant for presentation, because matrix multiplication was stack-based and could still be interpreted as post-multiplication, but, worse, reality leaked through the C-based API because individual elements would be accessed as `M[vector][coordinate]` or, effectively, `M[column][row]`, which unfortunately muddled the convention that the designer sought to adopt, and this was even preserved in the OpenGL Shading Language that was later added (although this also makes it possible to access coordinates by name instead, e.g., `M[vector].y`). As a result, many developers will now simply declare that having the column as the first index is the definition of column-major, even though this is clearly not the case with a real column-major language like Fortran.

Torch (for Lua) changed from column-major[23] to row-major[24] default order.

Transposition

As exchanging the indices of an array is the essence of array transposition, an array stored as row-major but read as column-major (or vice versa) will appear transposed. As actually performing this rearrangement in memory is typically an expensive operation, some systems provide options to specify individual matrices as being stored transposed. The programmer must then decide whether or not to rearrange the elements in memory, based on the actual usage (including the number of times that the array is reused in a computation).

For example, the Basic Linear Algebra Subprograms functions are passed flags indicating which arrays are transposed.[25]

Address calculation in general

The concept generalizes to arrays with more than two dimensions.

For a *d*-dimensional $N_1 \times N_2 \times \cdots \times N_d$ array with dimensions N_k (*k*=1...*d*), a given element of this array is specified by a tuple $(n_1, n_2, \ldots, n_d)$ of *d* (zero-based) indices $n_k \in [0, N_k - 1]$.

In row-major order, the *last* dimension is contiguous, so that the memory-offset of this element is given by:

$$n_d + N_d \cdot (n_{d-1} + N_{d-1} \cdot (n_{d-2} + N_{d-2} \cdot (\cdots + N_2 n_1) \cdots)) = \sum_{k=1}^{d} \left(\prod_{\ell=k+1}^{d} N_\ell \right) n_k$$

In column-major order, the *first* dimension is contiguous, so that the memory-offset of this element is given by:

$$n_1 + N_1 \cdot (n_2 + N_2 \cdot (n_3 + N_3 \cdot (\cdots + N_{d-1} n_d) \cdots)) = \sum_{k=1}^{d} \left(\prod_{\ell=1}^{k-1} N_\ell \right) n_k$$

where the empty product is the multiplicative identity element, i.e., $\prod_{\ell=1}^{0} N_\ell = \prod_{\ell=d+1}^{d} N_\ell = 1$.

For a given order, the stride in dimension *k* is given by the multiplication value in parentheses before index n_k in the right-hand side summations above.

More generally, there are *d!* possible orders for a given array, one for each permutation of dimensions (with row-major and column-order just 2 special cases), although the lists of stride values are not necessarily permutations of each other, e.g., in the 2-by-3 example above, the strides are (3,1) for row-major and (1,2) for column-major.

See also

- Array (data structure)
- Comparison of programming languages (array)
- Index origin, another difference between array types across programming languages
- Matrix representation
- Morton order, another way of mapping multidimensional data to a one-dimensional index, useful in tree data structures
- CSR format, a technique for storing sparse matrices in memory
- Vectorization (mathematics), the equivalent of turning a matrix into the corresponding column-major vector

References

1. "Cache Memory" (http://pld.cs.luc.edu/courses/264/spr19/notes/cache.html). *Peter Lars Dordal*. Retrieved 2021-04-10.
2. "Arrays and Formatted I/O" (https://www.fortrantutorial.com/arrays-formatted-io/index.php). *FORTRAN Tutorial*. Retrieved 19 November 2016.
3. "Why numbering should start at zero" (https://www.cs.utexas.edu/users/EWD/transcriptions/EWD08xx/EWD831.html). *E. W. Dijkstra Archive*. Retrieved 2 February 2017.
4. "Language Reference Version 4 Release 3" (https://www.ibm.com/support/knowledgecenter/SSQ2R2_9.0.0/com.ibm.ent.pl1.zos.doc/topics/lrm.pdf) (PDF). IBM. Retrieved 13 November 2017. "Initial values specified for an array are assigned to successive elements of the array in row-major order (final subscript varying most rapidly)."
5. "ISO/IEC 7185:1990(E)" (http://www.pascal-central.com/docs/iso7185.pdf) (PDF). "An array-type that specifies a sequence of two or more index-types shall be an abbreviated notation for an array-type specified to have as its index-type the first index-type in the sequence and to have a component-type that is an array-type specifying the sequence of index-types without the first index-type in the sequence and specifying the same component-type as the original specification."
6. Cohen, S.; Vincent, C. M. (1971-05-01). Introduction to SPEAKEASY (https://www.osti.gov/biblio/7303694) (Report). Argonne National Lab., IL (USA).
7. "SAS® 9.4 Language Reference: Concepts, Sixth Edition" (https://documentation.sas.com/api/docsets/lrcon/9.4/content/lrcon.pdf) (PDF). SAS Institute Inc. September 6, 2017. p. 573. Retrieved 18 November 2017. "From right to left, the rightmost dimension represents columns; the next dimension represents rows. [...] SAS places variables into a multidimensional array by filling all rows in order, beginning at the upper left corner of the array (known as row-major order)."
8. "Columns, Rows, and Array Majority" (https://www.nv5geospatialsoftware.com/docs/Columns__Rows__and_Array.html#:~:text=IDL%20and%20Fortran%20are%20both%20examples%20of%20column%2Dmajor%20languages.). *www.nv5geospatialsoftware.com*. Retrieved 31 July 2024.
9. MATLAB documentation, MATLAB Data Storage (https://www.mathworks.com/help/matlab/matlab_external/matlab-data.html#f22019) (retrieved from Mathworks.co.uk, January 2014).
10. "Multi-dimensional Arrays" (https://docs.julialang.org/en/v1/manual/arrays/). *Julia*. Retrieved 9 November 2020.
11. Spiegelhalter et al. (2003, p. 17): Spiegelhalter, David; Thomas, Andrew; Best, Nicky; Lunn, Dave (January 2003), "Formatting of data: S-Plus format", *WinBUGS User Manual* (https://web.archive.org/web/20030518074556/http://www.mrc-bsu.cam.ac.uk/bugs/winbugs/manual14.pdf) (PDF) (Version 1.4 ed.), Cambridge, UK: MRC Biostatistics Unit, Institute of Public Health, archived from the original (http://www.mrc-bsu.cam.ac.uk/bugs/winbugs/manual14.pdf) (PDF) on 2003-05-18
12. *An Introduction to R*, Section 5.1: Arrays (https://cran.r-project.org/doc/manuals/R-intro.html#Arrays) (retrieved March 2010).
13. "FFTs with multidimensional data" (https://wiki.scilab.org/howto/FFTMultidimensionalData). *Scilab Wiki*. Retrieved 25 November 2017. "Because Scilab stores arrays in column major format, the elements of a column are adjacent (i.e. a separation of 1) in linear format."
14. "Internal array representation in rasdaman" (https://doc.rasdaman.org/03_contributing.html#internal-array-representation). *rasdaman.org*. Retrieved 30 March 2025.
15. "Java Language Specification" (https://docs.oracle.com/javase/specs/jls/se7/html/jls-10.html). Oracle. Retrieved 13 February 2016.

16. "object Array" (https://www.scala-lang.org/api/current/#scala.Array$). *Scala Standard Library*. Retrieved 1 May 2016.
17. "The Python Standard Library: 8. Data Types" (https://docs.python.org/3.6/library/datatypes.html). Retrieved 18 November 2017.
18. "Vectors and Matrices" (https://reference.wolfram.com/language/tutorial/Lists.html#2534). *Wolfram*. Retrieved 12 November 2017.
19. "11.2 – Matrices and Multi-Dimensional Arrays" (https://www.lua.org/pil/11.2.html). Retrieved 6 February 2016.
20. "The N-dimensional array (ndarray)" (https://numpy.org/doc/stable/reference/arrays.ndarray.html). *SciPy.org*. Retrieved 3 April 2016.
21. "Eigen: Storage orders" (https://eigen.tuxfamily.org/dox/group__TopicStorageOrders.html). *eigen.tuxfamily.org*. Retrieved 2017-11-23. "If the storage order is not specified, then Eigen defaults to storing the entry in column-major."
22. "Column Vectors Vs. Row Vectors" (https://steve.hollasch.net/cgindex/math/matrix/column-vec.html). Retrieved 12 November 2017.
23. "Tensor" (https://torch5.sourceforge.net/manual/torch/index-6.html). Retrieved 6 February 2016.
24. "Tensor" (https://github.com/torch/torch7/blob/master/doc/tensor.md#storage-storage). *Torch Package Reference Manual*. Retrieved 8 May 2016.
25. "BLAS (Basic Linear Algebra Subprograms)" (https://www.netlib.org/blas/). Retrieved 2015-05-16.

Sources

- Donald E. Knuth, *The Art of Computer Programming Volume 1: Fundamental Algorithms*, third edition, section 2.2.6 (Addison-Wesley: New York, 1997).